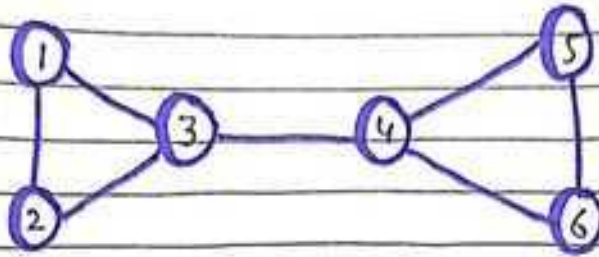


Subjective Type

Q No 3 (Graph Data Structure)



Adjacency Matrix :-

To	1	2	3	4	5	6
1	0	1	1	0	0	0
2	1	0	1	0	0	0
3	1	1	0	1	0	0
4	0	0	1	0	1	1
5	0	0	0	1	0	1
6	0	0	0	1	1	0

From To

(3 → 4)

From To

(4 → 6)

From

Adjacency List :-

Nodes	Neighbors
1	3, 2
2	3, 1
3	1, 2, 4
4	3, 5, 6
5	4, 6
6	4, 5

Q No. 01

// utility Function

```

void highestFrequency (node* root, unordered_map<int, int> &umap)
{
    if (root == nullptr)
    {
        return;
    }
    highestFrequency (root->left, umap);
    highestFrequency (root->right, umap);
    umap[root->data]++;
}

```

// Wrapper Function

```

int findHighestFrequency (unordered_map<int, int> &umap)
{
    int max = INT_MIN;
    int num;
    highestFrequency (root, umap);
    for (auto pair : umap)
    {
        if (max < pair.second)
        {
            max = pair.second;
            num = pair.first;
        }
    }
    return num;
}

```

Q #02

```

void pairOfNodes-(LinkedList list, int target)
{
    node* head = list; head;
    node* tail = list.tail;

    if(head == nullptr || tail == nullptr)
    {
        return;
    }
    node* temp = head;
    while (temp != nullptr)
    {
        node* j = temp->next;
        while (j != nullptr)
        {
            if (temp->data + j->data == target)
            {
                cout << "(" << temp->data << " , "
                    << j->data << ")" << endl;
            }
            else if (temp->data - j->data == target)
            {
                cout << "(" << temp->data << " - "
                    << j->data << endl;
            }
            j = j->next;
        }
        temp = temp->next;
    }
}

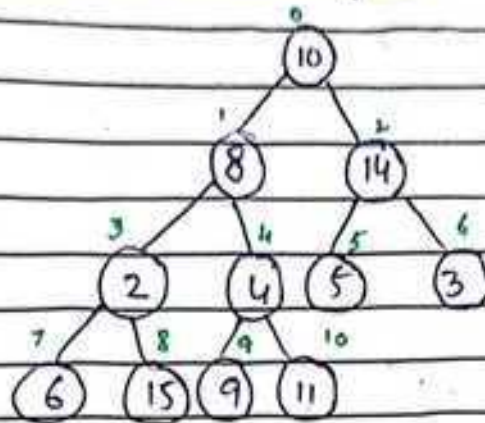
```

Q No. 6

```
int binarySearch(int arr[], int size, int key)
{
    int low = 0;
    int high = size - 1;
    while (low <= high)
    {
        int mid = (low + high) / 2;
        if (arr[mid] == key)
        {
            return key; mid
        }
        else if (arr[mid] < key)
        {
            low = mid + 1;
        }
        else if (arr[mid] > key)
        {
            high = mid - 1;
        }
    }
    return -1;
}
```

Q No 7 (Min Heap)

Step 1: (Build Heap)



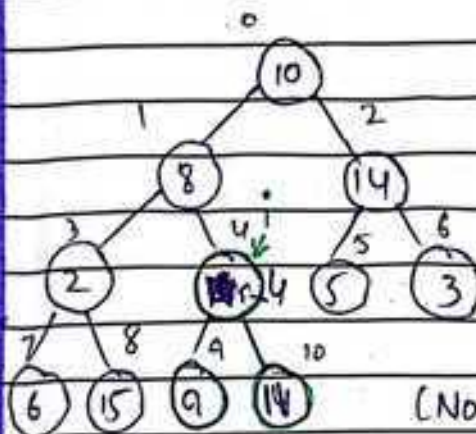
Calculate starting

$$i = \frac{n-1}{2}$$

$$i = \frac{11-1}{2}$$

$$i = 5-1$$

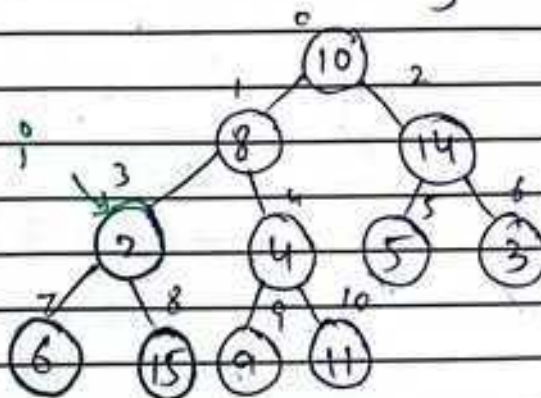
$$i = 4$$



(No Change)

Processing should start from index 4.

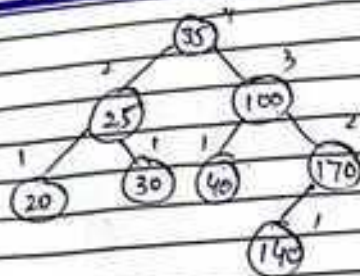
Now, decrement i by 1 and



(No change)

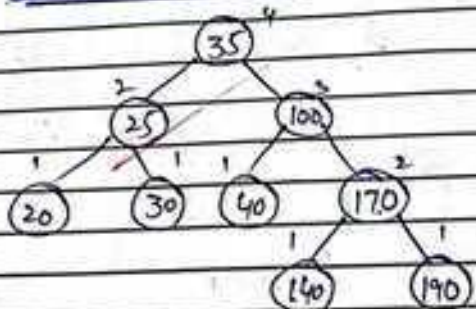
Now, decrement i by 1.

→ Insert 140



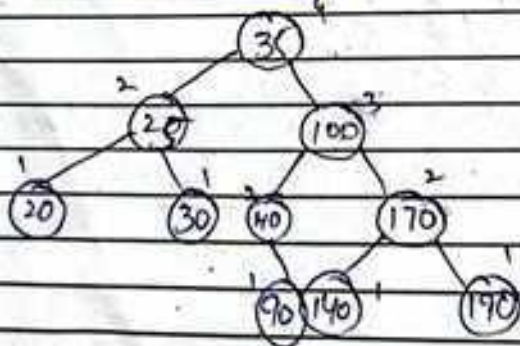
(Balanced)

→ Insert 190



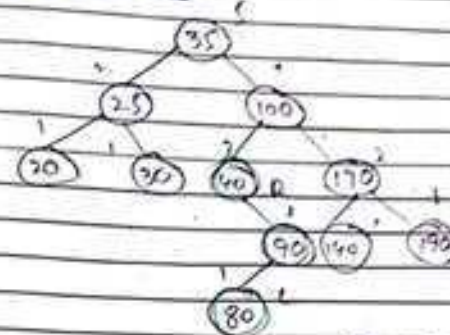
(Balanced)

→ Insert 90



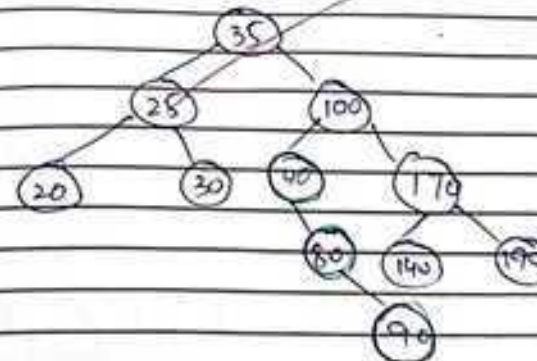
(Balanced)

→ Insert 80

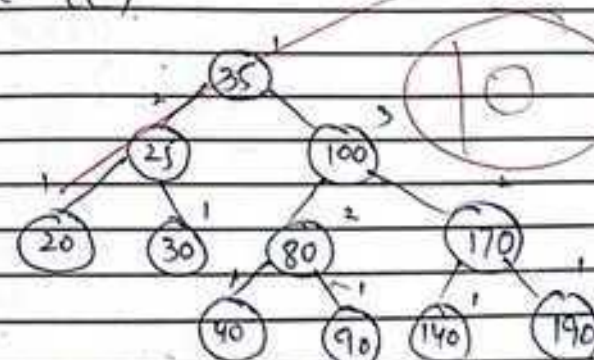


(Not Balanced)

Rotate RR,



Rotate LL,



This is the Final AVL after inserting All keys. (Balanced)